

pyAmpliCol 0.1.0

Methodology and Performance Report

Rikkert Frederix

Valentin Hirschi

Timea Vitos

Release 0.1.0

Abstract

This report defines the public theory, validation, and performance methodology for `pyAmpliCol`, release 0.1.0. It fixes the canonical process coverage for the built-in Standard Model, an externally loaded Standard Model, a scalar contact model, and a scalar-gravity model. The release result caches are intentionally reset: every table entry is `N/A`. Consequently this document makes no numerical performance claim. Its tables are a reproducible publication surface for measurements produced later with the released typed API and recorded with their effective configuration and environment.

Contents

1	Scope	2
2	Current Recursion and Model Lowering	2
2.1	Shared-current DAG	2
2.2	Contact terms and spin two	2
3	Colour and Runtime Semantics	3
4	Benchmark Methodology	3
4.1	Configuration contract	3
4.2	Numerical validation	3
5	Standard-Model Process Matrices	5
5.1	Built-in SM LC process matrix	5
5.2	Built-in SM NLC process matrix	6
5.3	Built-in SM full-colour process matrix	6
5.4	External SM LC process matrix	7
5.5	External SM NLC process matrix	8
5.6	External SM full-colour process matrix	8
6	Z-plus-Jets Evaluator Ladders	9
6.1	Built-in SM $d\bar{d} \rightarrow Z + (n-1)g$ ladder	9
6.2	External SM $d\bar{d} \rightarrow Z + (n-1)g$ ladder	9
7	Colour-Singlet Model Ladders	10
7.1	Scalar-contact ladder	10
7.2	Scalar-gravity ladder	10
8	Interpretation	10

1 Scope

pyAmpliCol generates tree-level scattering-amplitude artifacts and evaluates them through the Rusticol runtime. The Python layer resolves models and processes, constructs a shared-current directed acyclic graph (DAG), lowers bounded stages to Symbolica evaluators, and writes a schema-v3 process artifact. The runtime loads that artifact and provides summed and resolved evaluation through one physics contract to Python, Rust 2021, C++17, and Fortran 2008 clients.

The report has three purposes:

1. specify the observables and conditions required for a publishable benchmark;
2. preserve a stable matrix of representative process families and colour approximations; and
3. separate measured data from prose by rendering all result tables from validated JSON caches.

An N/A entry means that the standalone release has no recorded measurement for that slot. It is neither a zero, a failure, nor a value carried over from another implementation.

2 Current Recursion and Model Lowering

2.1 Shared-current DAG

For an external subset I , particle state t , and local spin or flow label χ , an off-shell current has the schematic recursion

$$J_{t,\chi}^\alpha(I) = P^\alpha_{\beta}(t, p_I) \sum_{I=A \dot{\cup} B} V^\beta_{\gamma\delta}(t_A, t_B \rightarrow t) J_{t_A, \chi_A}^\gamma(A) J_{t_B, \chi_B}^\delta(B), \quad p_I = \sum_{i \in I} p_i. \quad (1)$$

Current identity includes the particle, external labels, helicity ancestry, chirality or spin state, flavour and charge flow, colour state, momentum mask, and any auxiliary-current kind. Contributions with the same complete identity share one node. A topological sweep therefore evaluates each retained current once per phase-space point and reuses it in all dependent amplitudes.

External UFO or serialized JSON sources are compiled to the same canonical model representation as the process generator consumes. Particle metadata, interactions, coupling orders, Lorentz tensors, colour tensors, propagators, and runtime parameters come from that representation. UFO directories contain trusted executable Python; serialized JSON is the portable data-only source.

Optimizations on the external-model path are proof-gated rather than selected by particle names or PDG values. Exact normalized colour and Lorentz expressions, propagator metadata, and coupling relations certify current reflection, helicity, and reusable-kernel identities. For overlapping Standard-Model validation processes, the resulting current, interaction, amplitude-root, and colour-sector counts must match the built-in compatibility model after applying the same independently inferred perturbative-order envelope. An external compiler may retain fewer evaluator calls only when an additional exact kernel-equivalence certificate proves that reuse.

2.2 Contact terms and spin two

A vertex with valence above three is lowered to a deterministic tree of non-propagating auxiliary currents. The construction inserts the original coupling once and introduces no pole. Direct contraction of the original contact tensor is the validation oracle for the scalar-contact ladder.

The scalar-gravity ladder exercises rank-two external states and momentum-dependent interactions. For a massless graviton, the helicity-two polarization tensor is represented as

$$\epsilon_{\pm 2}^{\mu\nu} = \epsilon_{\pm}^\mu \epsilon_{\pm}^\nu. \quad (2)$$

Validation checks symmetry, transversality, tracelessness, normalization, and agreement between ordinary and higher-precision evaluation.

3 Colour and Runtime Semantics

The process matrices use three colour contracts:

LC	Leading-colour ordered amplitudes with physical colour-flow selectors.
NLC	The leading term plus the first retained $1/N_c^2$ -suppressed contribution.
Full	The complete supported sparse colour metric for the generated amplitude basis.

The optimized `Runtime.evaluate()` path returns one summed matrix element per point.

The resolved method `Runtime.evaluate_resolved()` exposes LC values with axes (`point`, `helicity`, `flow`). NLC and full-colour artifacts expose one already contracted colour component per helicity. Summing all non-point axes must reproduce the optimized total. Public selectors use physical helicity and flow identifiers; graph-local colour-sector identifiers are not report inputs.

All language bindings load the same process manifest and evaluator payloads. Cross-language agreement therefore validates the runtime interfaces and data layout. A separately generated reference is required for an independent physics comparison.

The default JIT artifact embeds a self-contained SymJIT application for f64 evaluation. Rusticol executes that payload through the separate MIT-licensed SymJIT runtime without importing Symbolica or consulting its runtime license state. Higher-precision Python evaluation and artifacts generated with the ASM or C++ evaluator backends retain an explicit Symbolica runtime capability.

4 Benchmark Methodology

4.1 Configuration contract

Each measurement begins from an immutable resolved configuration. Unless a table variant says otherwise, the report contract uses the release defaults: requested SymJIT optimization level 3, evaluator batch size 128, output chunk size 128, stage-local parameter layout, ten Horner iterations, backend-default common-pair elimination, a ten-second benchmark target, two warmup runs, and at least five timed samples. The process matrices always record requested level 3 in their machine-readable benchmark contract; only explicitly labelled comparison rows, such as JIT level 1 in the dedicated *Z* ladder, override it. SymJIT may apply its documented backend fallback independently of the requested level. Requested and effective configurations are stored separately so resource or license adjustments remain visible.

Generation time is wall time for one explicit `Generator.generate()` operation starting from the declared model source and ending with a committed, loadable artifact. Model-source conversion, process generation, evaluator compilation, and API-bundle emission must be identified separately when they are measured separately. Existing artifacts may be benchmarked, but their generation slot remains N/A unless a matching generation measurement is available.

Runtime measurements use `BenchmarkRunner` and the summed `Runtime.evaluate()` observable. A cache entry records sample count, wall and evaluator time per point, standard deviation, standard error, relative standard error, effective settings, and environment. Comparisons are valid only for the same process, colour accuracy, parameters, phase-space ensemble, precision, selectors, and normalization.

4.2 Numerical validation

The matrix element and its independent reference are evaluated at the same deterministic points with the same parameter card. Shared-core language checks use relative tolerance 10^{-12} and absolute tolerance 10^{-15} . Independent reference comparisons use relative tolerance 10^{-8} , with higher precision used to diagnose unstable points. A populated result must also satisfy the resolved-sum identity and preserve expected structural zeros.

No timing is accepted as a numerical validation result. Conversely, a valid matrix element does not populate generation or runtime fields without a matching timing run.

Publication workflow. The report service validates all cache entries, renders every table in a staging directory, optionally compiles the PDF there, and publishes the JSON, TeX, and PDF set only after all steps

succeed. Failed refreshes leave the previous published set in place. Physics generation and benchmarking are not implicit report operations.

5 Standard-Model Process Matrices

The canonical matrix spans electroweak vector production, charged-current and leptonic channels, heavy-quark production, pure gluons, multi-boson final states, Higgs-associated final states, and three- and four-quark-line cases. Here n is the number of final-state particles. LC coverage retains the established grid through $n = 9$, with the four bounded families ending at $n = 8$. NLC and full-colour grids retain $n = 1, \dots, 5$; cells below a family’s minimum physical multiplicity remain explicitly marked N/A and carry an applicability flag in JSON.

The external-SM matrices use the same process definitions and colour contracts as the built-in matrices. Their model source is the packaged Standard Model loaded through the external JSON/UFO compilation path, so the comparison isolates model-source and generic-lowering behaviour without changing the requested physics.

5.1 Built-in SM LC process matrix

ID	process family	$n = 1$	$n = 2$	$n = 3$	$n = 4$	$n = 5$	$n = 6$	$n = 7$	$n = 8$	$n = 9$
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

A populated cell reports generation time and summed wall time per phase-space point. Release cells are intentionally N/A; applicability and independent-reference slots remain in JSON.

5.2 Built-in SM NLC process matrix

ID	process family	$n = 1$	$n = 2$	$n = 3$	$n = 4$	$n = 5$
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A	N/A	N/A

5.3 Built-in SM full-colour process matrix

ID	process family	$n = 1$	$n = 2$	$n = 3$	$n = 4$	$n = 5$
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A	N/A	N/A

A populated cell reports generation time and summed wall time per phase-space point. Release cells are intentionally N/A; applicability and independent-reference slots remain in JSON.

5.4 External SM LC process matrix

ID	process family	$n = 1$	$n = 2$	$n = 3$	$n = 4$	$n = 5$	$n = 6$	$n = 7$	$n = 8$	$n = 9$
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

A populated cell reports generation time and summed wall time per phase-space point. Release cells are intentionally N/A; applicability and independent-reference slots remain in JSON.

5.5 External SM NLC process matrix

ID	process family	$n = 1$	$n = 2$	$n = 3$	$n = 4$	$n = 5$
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A	N/A	N/A

5.6 External SM full-colour process matrix

ID	process family	$n = 1$	$n = 2$	$n = 3$	$n = 4$	$n = 5$
1	$d\bar{d} \rightarrow Z + (n-1)g$	N/A	N/A	N/A	N/A	N/A
2	$u\bar{d} \rightarrow W^+ + (n-1)g$	N/A	N/A	N/A	N/A	N/A
3	$d\bar{d} \rightarrow e^+e^- + (n-2)g$	N/A	N/A	N/A	N/A	N/A
4	$u\bar{d} \rightarrow e^+\nu_e + (n-2)g$	N/A	N/A	N/A	N/A	N/A
5	$d\bar{d} \rightarrow ZZ + (n-2)g$	N/A	N/A	N/A	N/A	N/A
6	$gg \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A
7	$d\bar{d} \rightarrow t\bar{t} + (n-2)g$	N/A	N/A	N/A	N/A	N/A
8	$gg \rightarrow gg + (n-2)g$	N/A	N/A	N/A	N/A	N/A
9	$d\bar{d} \rightarrow ZZZ + (n-3)g$	N/A	N/A	N/A	N/A	N/A
10	$d\bar{d} \rightarrow e^+e^-ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A
11	$d\bar{d} \rightarrow t\bar{t}ZH + (n-4)g$	N/A	N/A	N/A	N/A	N/A
12	$d\bar{d} \rightarrow e^+e^-e^+e^- + (n-4)g$	N/A	N/A	N/A	N/A	N/A
13	$d\bar{d} \rightarrow u\bar{u}s\bar{s} + (n-4)g$	N/A	N/A	N/A	N/A	N/A
14	$d\bar{d} \rightarrow u\bar{u}s\bar{s}c\bar{c} + (n-6)g$	N/A	N/A	N/A	N/A	N/A

A populated cell reports generation time and summed wall time per phase-space point. Release cells are intentionally N/A; applicability and independent-reference slots remain in JSON.

6 Z-plus-Jets Evaluator Ladders

The dedicated $d\bar{d} \rightarrow Z + (n-1)g$ ladders compare the independent reference with JIT level 1, JIT level 3, ASM O3, and C++ O3 evaluator configurations. They retain final-state multiplicities $n = 1, \dots, 9$ for the built-in model and $n = 1, \dots, 6$ for the external-model campaign. Variant overrides are stored as typed dotted configuration paths in the cache rather than inferred from table labels.

6.1 Built-in SM $d\bar{d} \rightarrow Z + (n-1)g$ ladder

evaluator	$n = 1$	$n = 2$	$n = 3$	$n = 4$	$n = 5$	$n = 6$	$n = 7$	$n = 8$	$n = 9$
Independent reference	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
JIT level 1	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
JIT level 3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
ASM O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
C++ O3	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

6.2 External SM $d\bar{d} \rightarrow Z + (n-1)g$ ladder

evaluator	$n = 1$	$n = 2$	$n = 3$	$n = 4$	$n = 5$	$n = 6$
Independent reference	N/A	N/A	N/A	N/A	N/A	N/A
JIT level 1	N/A	N/A	N/A	N/A	N/A	N/A
JIT level 3	N/A	N/A	N/A	N/A	N/A	N/A
ASM O3	N/A	N/A	N/A	N/A	N/A	N/A
C++ O3	N/A	N/A	N/A	N/A	N/A	N/A

Cells use the matrix generation-time and summed-runtime convention. Variant overrides are machine-readable in JSON.

7 Colour-Singlet Model Ladders

The scalar-contact ladder retains $n = 2, \dots, 8$ final scalars. It tests generic high-valence contact decomposition, identical-particle normalization, and source parity without Standard-Model-specific kernels. The scalar-gravity ladder retains $n = 2, \dots, 4$ final gravitons and exercises spin-two wavefunctions, tensor propagators, and momentum-dependent vertices. Since all external particles in these fixtures are colour singlets, LC, NLC, and full colour reduce to the same unit colour contraction.

7.1 Scalar-contact ladder

	scalar_0 scalar_0 > X*scalar_0						
metric	$X = 2$	$X = 3$	$X = 4$	$X = 5$	$X = 6$	$X = 7$	$X = 8$
status	N/A	N/A	N/A	N/A	N/A	N/A	N/A
generation [s]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
wall [s/pt]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
evaluator [s/pt]	N/A	N/A	N/A	N/A	N/A	N/A	N/A
matrix element	N/A	N/A	N/A	N/A	N/A	N/A	N/A
relative difference	N/A	N/A	N/A	N/A	N/A	N/A	N/A

7.2 Scalar-gravity ladder

	scalar_0 scalar_0 > X*graviton		
metric	$X = 2$	$X = 3$	$X = 4$
status	N/A	N/A	N/A
generation [s]	N/A	N/A	N/A
wall [s/pt]	N/A	N/A	N/A
evaluator [s/pt]	N/A	N/A	N/A
matrix element	N/A	N/A	N/A
relative difference	N/A	N/A	N/A

8 Interpretation

This release establishes coverage and methodology, not a baseline. Performance regression gates should be enabled only after the same release artifact has produced a complete, reviewed baseline on named hardware. Until then, consumers should treat every N/A entry as unavailable and should not interpolate, copy, or estimate a value from neighbouring multiplicities or model profiles.

The machine-readable caches under `docs/results/` are authoritative for table completeness and measurement provenance. The TeX inputs and this PDF are derived publication artifacts.